Data Engineering Technical Assignment: Candidate Brief

Overview

You will build a small, production-minded data pipeline on public survey data from the General Social Survey (GSS), model it with dbt, query it with SQL, and document your decisions. Expect about **8 hours** of work and submit within **7 days**.

- **What you build:** two data extracts, a Python pipeline that loads them, a dbt model, analytical SQL, a reproducible and orchestrated environment, and a short report.
- **How it is judged:** this assignment is a critical part of our evaluation. Section 6 explains how it is assessed and what happens next.
- **Bonus items:** anything marked *bonus* earns extra credit and is never penalised if omitted.

1. Dataset and Data Extract

You will work with the **General Social Survey (GSS)**, a long-running nationally representative US survey run by NORC. Access it through the **GSS Data Explorer** (gssdataexplorer.norc.org). Your repository, including the committed extracts, may be public, but you must **cite NORC as the source of the data** in your README, using the citation specified in the Data Explorer's terms and conditions.

The data you collect must be able to answer the following questions about each respondent:

- What is the respondent's working status?
- If working, have they worked over 12 months in their lifetime?
- What is the respondent's education level?
- What industry does the respondent work in?
- How satisfied are they with their line of work?
- Do they believe in life after death?
- How frequently does the respondent pray?
- Which religion was the respondent raised in?
- How often does the respondent read a newspaper?
- How often does the respondent watch TV?
- Is the respondent interested in technology?

Beyond what those questions require, scope your work to respondent-level data on religiosity and media consumption, including at minimum: religious preference (`RELIG`), service attendance (`ATTEND`), strength of affiliation (`RELITEN`), hours of television per day (`TVHOURS`), weekly web use (`WWWHR`), the survey year, a sample weight, and the demographics you judge necessary (e.g. age, sex, education, region).

Requirements

- **Variable mapping.** Identify the GSS variables that best answer each question yourself. Do not assume one variable per question is always sufficient or always correct.
- **Size of extract.** You will build **two extracts (cuts)**, each a separate extract from the Data Explorer. Each should contain roughly **5,000 respondent records**, and together they must contain no more than **10,000 distinct respondents**. Choose your survey years deliberately and justify the choice. Section 2.2 sets out how the two cuts must relate.
- **Working with limitations.** Some questions above are asked only in certain years, on certain ballots, or of certain respondents, so your extracts may fall short of what you would ideally want for some of them. Do not hide this. Where they fall short, work within the limitation and justify your approach in your report. Where you can identify mitigation approaches, propose them and implement them.
- **Singular form.** Data is collected as **one record per respondent**, so that aggregate queries (counts, rates, cross-tabulations) can be run across respondents.
- **Export.** Build the extracts in the Data Explorer, **export them as JSON**, and commit them to the repository under `/source/data/raw/` .

Data profile (complete before building your pipeline)

Once you have your extracts, and before you build anything else, write a short data profile covering: record counts per year, the share of missing values per variable per year, and how the survey's special missing-value codes are distinguished from genuine answers. Your profile must state which variables are only asked in some years or on some ballots, and how that affects what questions your pipeline can honestly answer.

In your report (Section 5.1), also document which variable you chose for each question above, why, which years it covers, and how you handled records where a question did not apply or was not asked.

2. Pipeline and Data Model

Build a Python-based pipeline that takes the extracts you committed under `/source/data/raw/` and turns them into a modelled, queryable dataset that can answer the questions in Section 1 and support aggregate analysis across respondents.

2.1 Plan first

Before writing pipeline code, write a short design note (in `/docs`) stating: the grain of your core table(s), how a respondent is uniquely identified, the layers or stages you intend to use and why, your database choice and why, and the assumptions you are making about the data. Your final solution may diverge from

this note. If it does, say where and why.

2.2 Ingestion and load (Python)

- Provide **two extracts (cuts)**, each of roughly 5,000 respondent records. The cuts must **not be identical**: each must include survey years the other does not. They must also **share at least 5% of their respondents**. Together they must contain **no more than 10,000 distinct respondents**. Document how you built them and why.
- Read the raw extracts, validate what you read, and load them into a database of your choice that dbt supports. Justify the choice and document the setup in your design note. Any database is acceptable.
- Raw data must be preserved as received. Transformations happen downstream of it, never in place.
- Your pipeline must be **safe to re-run**: running it twice, or loading the cuts in either order, must not duplicate or corrupt data. Show that it holds, including across the overlap.
- Handle bad or unexpected input deliberately (e.g. malformed records, unexpected values, schema surprises) and say how.

2.3 Modelling and transformation (dbt)

- Use **dbt** to build the transformation layers from your raw data to analysis-ready models.
- Your models must represent the survey's special missing-value codes so that "did not apply," "don't know," "no answer" and "not asked that year" cannot be mistaken for each other or for a genuine answer.
- Coded survey answers must be translated into something an analyst can read without opening the codebook.
- **Capture semantic metadata for the encodings.** For each coded variable, record in a machine-readable, version-controlled form what the codes mean, what kind of value each represents (e.g. a genuine answer versus a missing-data type), and where the variable is valid. Your models should use this metadata, not duplicate it.
- **Naming and conventions matter across the whole dbt layer.** Choose a consistent naming convention for models, columns, sources, seeds, tests, schemas and folders. Document it, and apply it everywhere. Reviewers will read your project as an analyst joining your team would.
- Design for the questions in Section 1, but do not hard-code your model to only those questions. A new analyst should be able to add another variable without rebuilding your model.
- Include dbt **tests** that encode real expectations about this data (not just defaults), and dbt **documentation** for your models and columns.

2.4 Weights (bonus)

The GSS is a weighted sample. *Optional:* decide which weight your models carry, how an analyst should use it, and justify the choice.

3. Analysis and Efficiency

3.1 Analytical SQL

Using your dbt models, write SQL that answers each of the following. Keep the queries in your repository (for example as dbt analyses, or in a folder of your choice documented in your README). For each one, state your result and any caveats about how far it can be trusted.

1. How is prayer frequency distributed among respondents, by the religion they were raised in?
2. What share of respondents believe in life after death, by survey year? Show how many respondents were not asked alongside the result.
3. For each education level, which industry is most common among working respondents?
4. Is time spent watching TV related to job satisfaction? Report group sizes alongside any averages or rates.
5. Among respondents who are not currently working, what share have ever worked for 12 months or more?
6. How have newspaper reading and TV viewing changed from one survey wave to the next?

You may not be able to answer every question honestly from your extracts. Where that is the case, say so, explain why, and show what you did instead.

Also provide one query that audits the overlap between your two cuts: the number of respondents in each cut, the number in both, the number of distinct respondents overall, and the overlap as a share of each cut. Show that your other results count each respondent once.

3.2 Efficiency

Your extracts are small, but the full GSS cumulative file has roughly 75,700 respondents and nearly 7,000 variables. In your report:

- Justify the materialization strategy for each layer of your dbt project (view, table, incremental or other), and say what would change at full scale.
- Show the query plan for your most expensive query, make one change that improves it, and show the difference. If your data is too small for a difference to show, explain how you would demonstrate it at scale.
- Explain how your load and your models behave when a new extract is added: what is reprocessed, and what is not.
- Say what you deliberately did **not** build because it would be disproportionate for this data volume.

3.3 Query quality

Your SQL will be read by another analyst. It should be readable, consistently formatted, and follow the naming conventions from Section 2.

4. Orchestration and Environment

4.1 Reproducibility

A reviewer must be able to run your pipeline end to end on a clean machine by following your README. The **core pipeline must run locally without paid cloud resources**. Package your environment accordingly. Docker, AWS-native services, or infrastructure as code with Terraform are all reasonable starting points, and you may choose another approach. Whatever you choose, justify it and pin your dependencies so that a run next month behaves like a run today.

4.2 Orchestration

Define how the pipeline's steps run, in what order, and what happens when one fails. At a minimum, cover: loading raw data, building and testing the dbt models, and generating documentation. Choose an orchestration approach appropriate to the scale of this data and justify it. Explain:

- what triggers a run, and what happens when a new extract is added
- how a failed run is detected and how it is safely re-run
- how a failing data test affects downstream steps

4.3 Configuration and secrets

No credentials or environment-specific settings may be committed to the repository. Show how configuration is supplied and how a new user sets it up.

4.4 Observability

Your pipeline must leave enough evidence behind for someone to tell what ran, what it processed, and why it failed. Say what you log and where.

4.5 Production sketch and extensibility (bonus)

Optional: describe, or provide as code, how this would run in AWS at full scale: where data is stored, what runs the transformations, what schedules them, how permissions are scoped, and what would drive cost. If you provide Terraform, it must at least validate.

Also describe how your design would take in **other relevant data sources** beyond the GSS (for example other surveys or external datasets on religiosity or media consumption). Say which sources you would add and why, and how each would be ingested, modelled alongside the GSS data, and kept consistent with it. You are free to make your own assumptions about these sources (volume, format, refresh cadence, licensing, access method). State them clearly.

4.6 Continuous integration (bonus)

Optional: configure continuous integration so that your dbt tests run automatically on every commit, and explain what a failing run blocks.

5. Report and Documentation

5.1 Report

Write a short report documenting your approach, saved under `/report`. It must be **no more than 5 pages**. Appendices are optional and at your discretion, and do not count toward the 5 pages. We prefer them light, specific and minimal in verbosity. The report should include:

- **Extract and variable mapping.** How you built your two cuts and why you chose those years. For each question in Section 1, which variable(s) you used, why, which years they cover, and how you handled records where a question did not apply or was not asked.
- **Data profile.** The profile described in Section 1.
- **Database and schema.** Your choice of database and the schemas or layers you used, with justification.
- **Model and structures.** A summary of your data model: grain, keys, layers, how encodings and missing values are represented, and the naming convention.
- **Libraries and frameworks.** What you used and why, including what you considered and rejected.
- **Results and caveats.** Your answers to the questions in Section 3 and how far each can be trusted.
- **Limitations and mitigations.** Where your extracts fell short, what you did about it, and what remains unsolved.
- **Risk and complexity analysis.** Assume the amount of data and the number of sources will grow substantially. Identify what breaks first, what grows fastest, and what you would change. Quantify where you can.
- **Assumptions and divergence.** Assumptions you made, and where your final build differs from your design note and why.
- **AI use and human oversight.** Disclose which AI tools you used and for what. Give an honest self-evaluation of the extent of AI contribution versus your own, and describe where a human was in the loop: what you reviewed, verified, corrected or rejected. AI assistance is permitted. You are assessed on your judgement and on the accuracy of this account.

5.2 README

Your repository must include a `README.md` with setup and run instructions, how to run the tests, a map of the repository, how to reproduce your extracts, and a **citation of NORC as the source of the data**.

5.3 Code and model documentation

Comment your code where the reason is not obvious from the code. Your dbt documentation requirements are in Section 2.3.

6. Submission and Evaluation

6.1 Repository structure

Organise your repository as follows:

- `/source` : your pipeline code and your data (extracts under `/source/data/raw/`)
- `/dbt` : your dbt project
- `/docs` : your design note (Section 2.1) and your naming conventions (Section 2.3)
- `/report` : your report (Section 5.1)
- `README.md` at the root

6.2 Effort

We expect this to take around **8 hours**. Please do not spend significantly more. A well-reasoned partial solution with honest notes about what is missing is better than an over-built one. State in your report roughly how long you spent, and what you would do with more time.

6.3 Submission

Submit the link to your repository within **7 days** of receiving this assignment. The repository may be public. It must cite NORC as the source of the data (Section 5.2).

6.4 How you will be assessed

This assignment is a critical part of our evaluation. Treat it with the seriousness of work you would put your name to.

Your submission will be assessed on: problem solving, planning, data modelling, ETL, dbt, SQL, efficiency, orchestration, and documentation. Items marked *bonus* earn extra credit and are never penalised if omitted. Proportionality counts: we value solutions sized to the problem.

Candidates whose submissions progress will be invited to a **panel interview in which you will be asked to defend your submission**: your decisions, trade-offs, results, and your use of AI tools. You should be able to explain every part of what you submit.